

AWS PORTFOLIO PROJECT

High-Availability Web Application

on Amazon Web Services

Multi-AZ Architecture | Auto Scaling | Application Load Balancer | RDS Multi-AZ

Bernard Tetteh

AWS Cloud Portfolio | May 2026

1. Project Overview

This project demonstrates the design and deployment of a production-grade, fault-tolerant web application on Amazon Web Services. The architecture targets high-availability use cases such as Electronic Health Record (EHR) systems, with a design goal of 99.99% uptime. All infrastructure was built and configured manually through the AWS Management Console.

1.1 Business Objectives

- Achieve high availability across multiple AWS Availability Zones
- Automatically recover from individual instance or entire AZ failures
- Distribute incoming traffic evenly across healthy instances via load balancing
- Provide a scalable managed relational database with automatic failover
- Implement defense-in-depth security with layered network controls

1.2 Architecture Summary

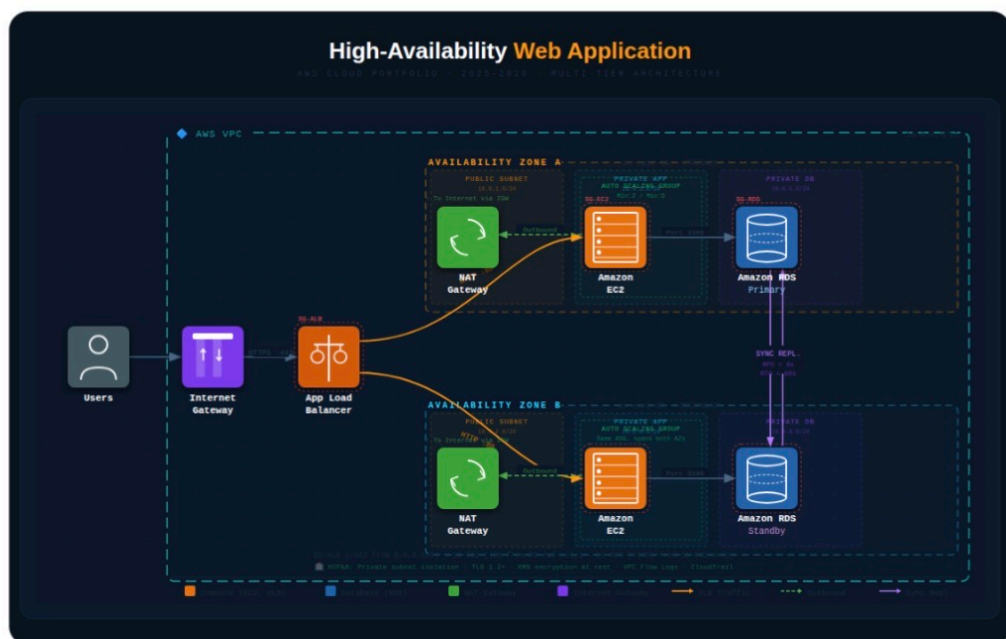
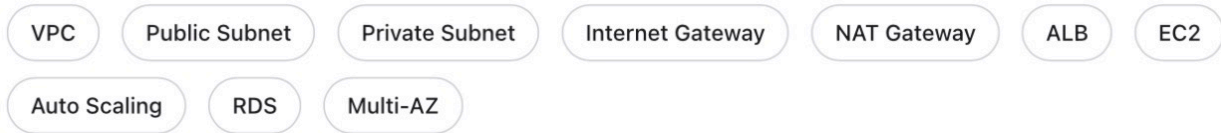
Specification	Value
Availability Zones	2 (us-east-1a, us-east-1b)
EC2 Instances (desired)	2 -- one per AZ
EC2 Instances (max)	4
Instance Type	t2.micro (Amazon Linux 2023)
Web Server	Apache httpd (auto-installed via User Data)
Database	RDS MySQL 8.4 -- Multi-AZ
Load Balancer	Application Load Balancer (internet-facing)
VPC CIDR	10.0.0.0/16
Region	us-east-1 (N. Virginia)

1.3 Architecture Diagram

The diagram below illustrates the complete multi-AZ architecture. Users reach the application through the Internet Gateway and Application Load Balancer. Traffic is distributed to EC2 instances in private subnets across both Availability Zones. NAT Gateways allow private instances outbound internet access. The RDS primary in AZ-A replicates synchronously to a standby in AZ-B for automatic failover.

High-Availability Web App In Progress

Multi-AZ architecture with Application Load Balancer, Auto Scaling EC2 instances, and Multi-AZ RDS for automated failover.



High-availability web application architecture using VPC with multi-AZ subnets, Application Load Balancer for traffic distribution, Auto Scaling EC2 instances, and Multi-AZ RDS for automated database failover.

Figure 1 -- High-Availability Web Application: VPC with two AZs, ALB, NAT Gateways, Auto Scaling EC2, and RDS Multi-AZ with synchronous replication

Architecture Layers

- Internet Edge: Internet Gateway + Application Load Balancer (public subnets)
- AZ-A Public Subnet: NAT Gateway for private instance outbound internet access
- AZ-A Private Subnet: EC2 instance (Auto Scaling) + RDS Primary (MySQL)
- AZ-B Public Subnet: NAT Gateway (redundant -- for full HA)
- AZ-B Private Subnet: EC2 instance (Auto Scaling) + RDS Standby (sync replication)

2. VPC & Networking Configuration

2.1 VPC

The HA-WebApp-VPC is the isolated network boundary for all project resources. It spans two Availability Zones with four subnets -- two public and two private -- providing full AZ-level redundancy.

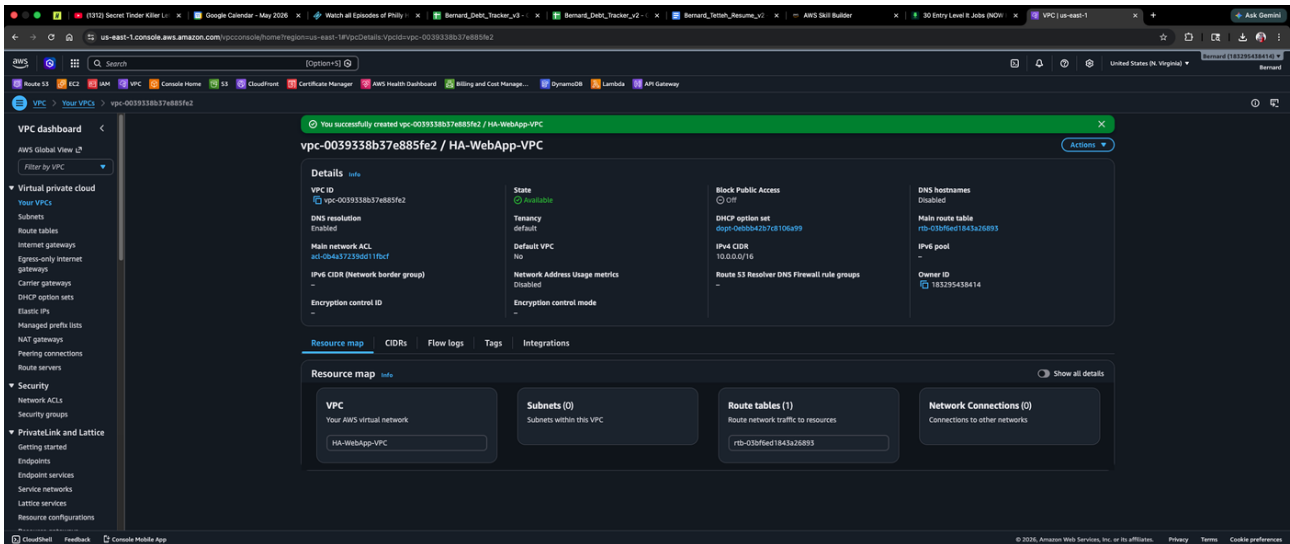


Figure 2 -- HA-WebApp-VPC: State=Available, IPv4 CIDR=10.0.0.0/16, DNS resolution enabled

2.2 Subnets

Four subnets are provisioned across both AZs. Public subnets host the ALB and NAT Gateway. Private subnets host EC2 instances and RDS -- no direct internet exposure.

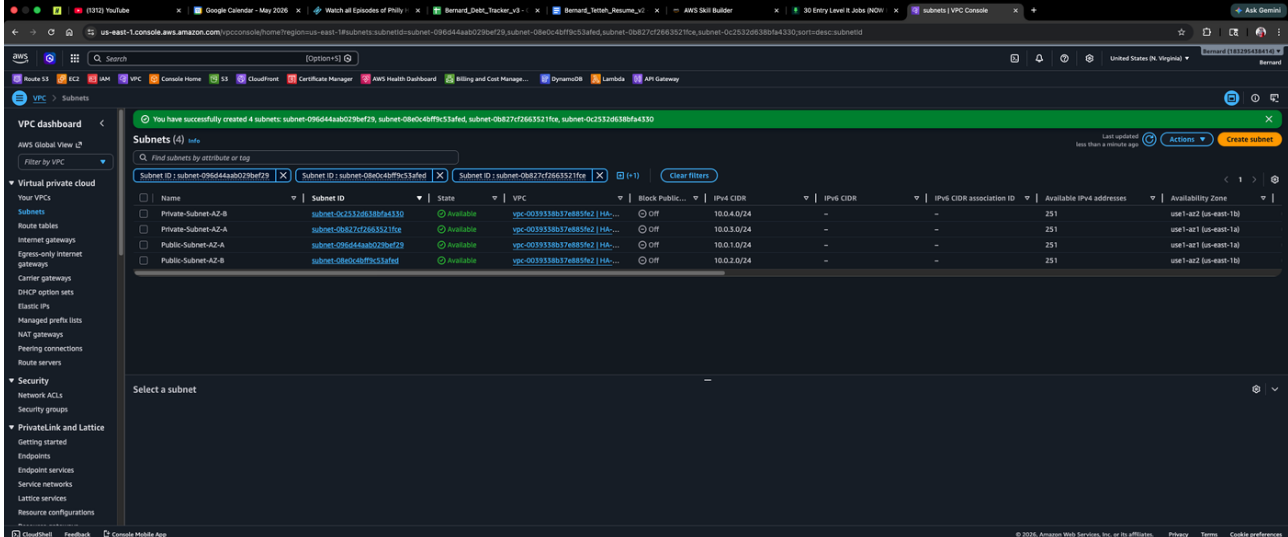


Figure 3 -- All 4 subnets created: Public-AZ-A (10.0.1.0/24), Public-AZ-B (10.0.2.0/24), Private-AZ-A (10.0.3.0/24), Private-AZ-B (10.0.4.0/24)

2.3 Internet Gateway

The Internet Gateway (HA-WebApp-IGW) is the VPC's entry point for all inbound internet traffic. It is attached to HA-WebApp-VPC and referenced in the public route table.

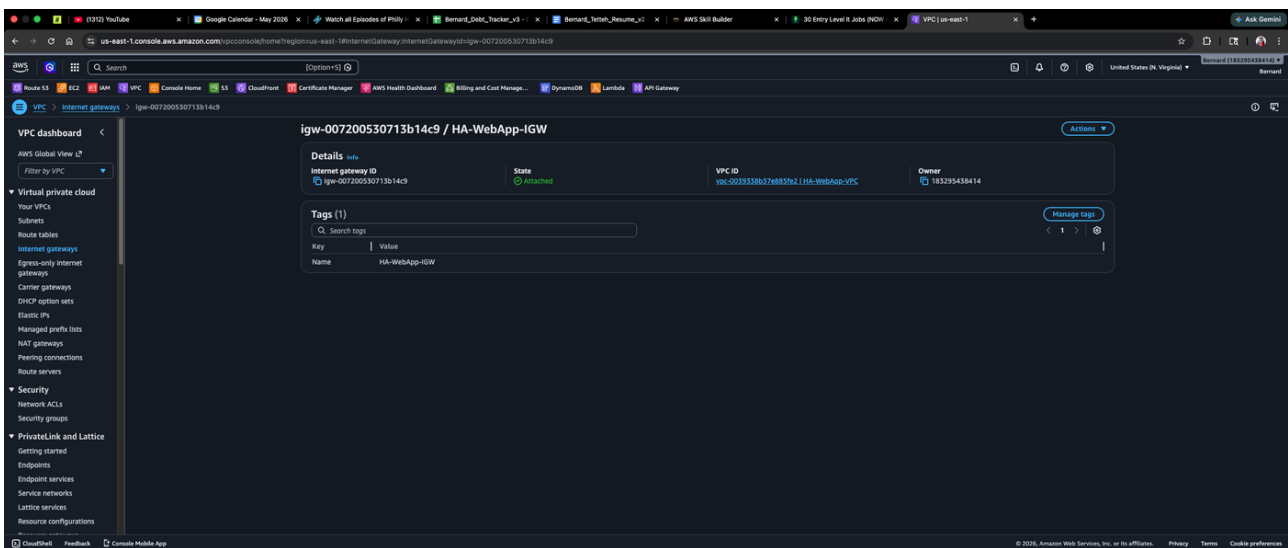


Figure 4 -- HA-WebApp-IGW: State=Attached, linked to vpc-0039338b37e885fe2 / HA-WebApp-VPC

2.4 Route Tables

Public Route Table -- internet traffic routed via Internet Gateway

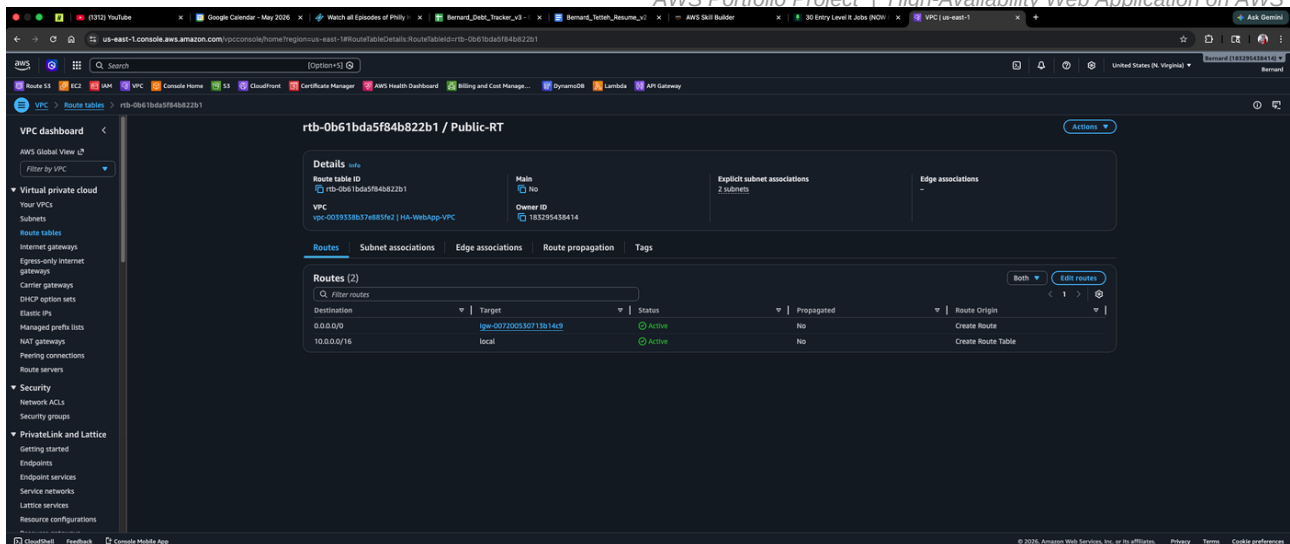


Figure 5 -- Public-RT Routes: 0.0.0.0/0 target=IGW (Active), 10.0.0.0/16 target=local (Active)

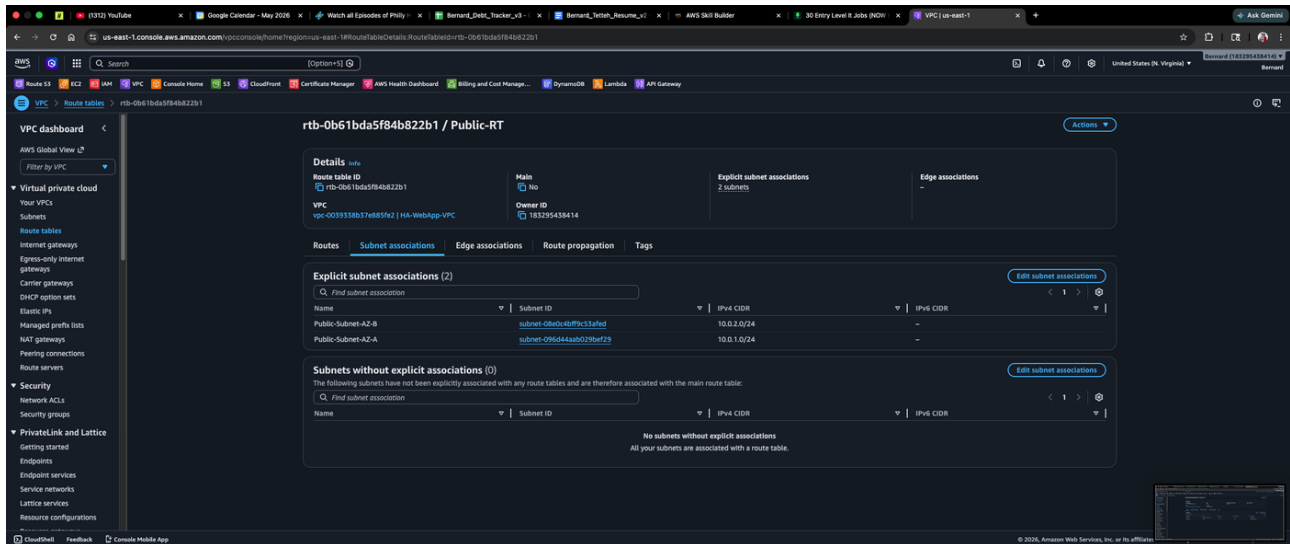


Figure 6 -- Public-RT Subnet Associations: Public-Subnet-AZ-A (10.0.1.0/24) and Public-Subnet-AZ-B (10.0.2.0/24)

Private Route Table -- outbound traffic routed via NAT Gateway

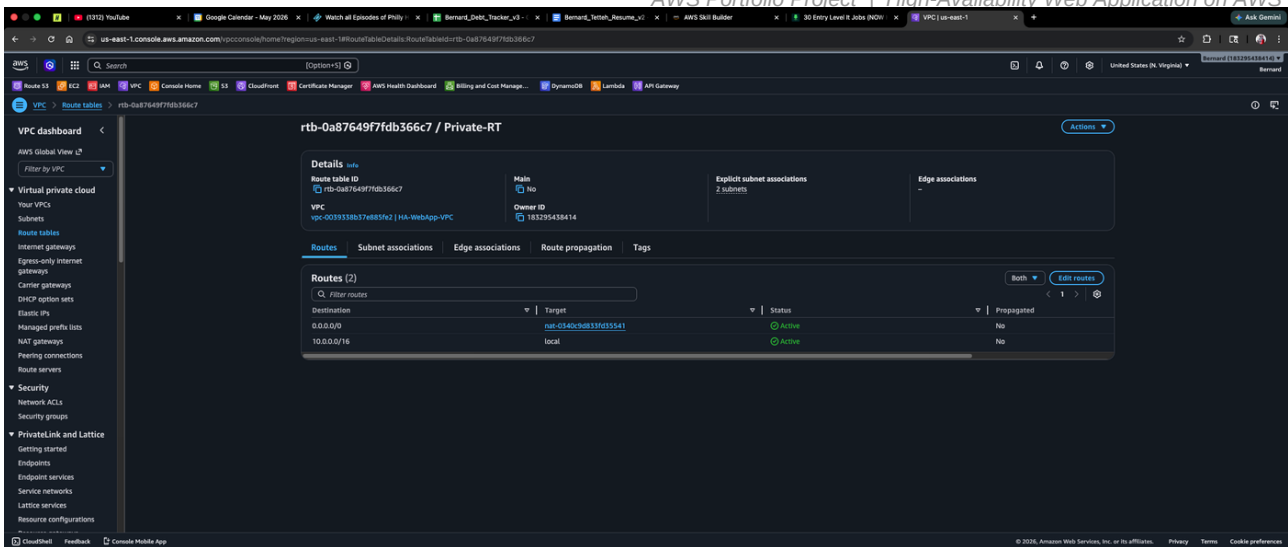


Figure 7 -- Private-RT Routes: 0.0.0.0/0 target=NAT Gateway (Active), 10.0.0.0/16 target=local (Active)

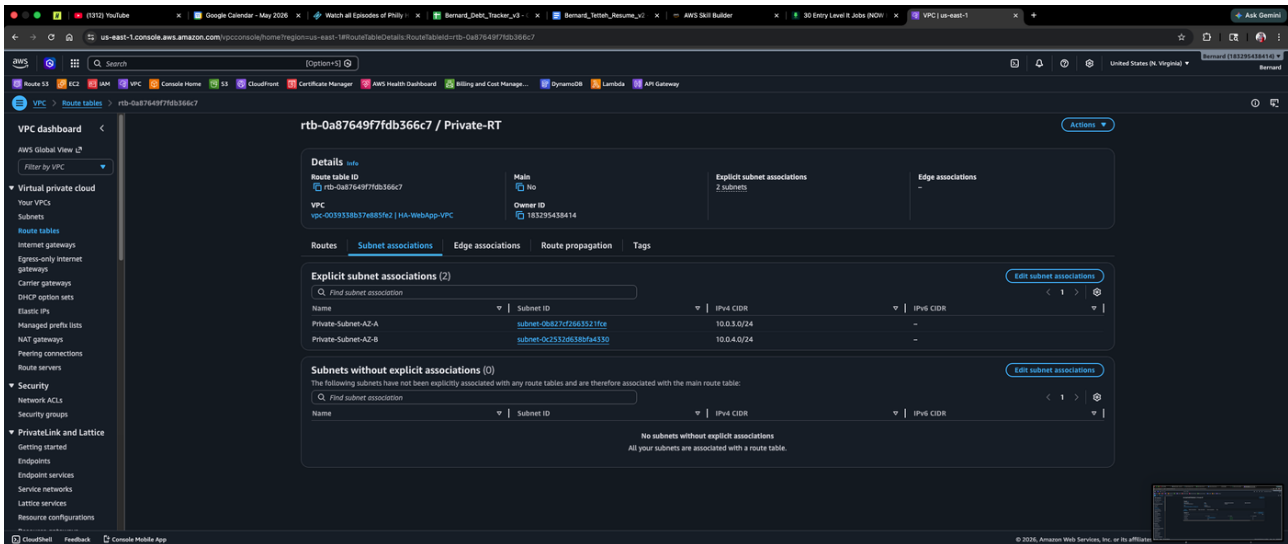


Figure 8 -- Private-RT Subnet Associations: Private-Subnet-AZ-A (10.0.3.0/24) and Private-Subnet-AZ-B (10.0.4.0/24)

3. Security Configuration

Security is implemented using a layered approach -- each tier has its own dedicated security group with minimum required permissions. No resource is directly reachable from the internet except through the ALB.

3.1 ALB Security Group (ALB-SG)

The ALB-SG allows internet users to reach the load balancer on standard web ports only. All backend resources are hidden behind it.

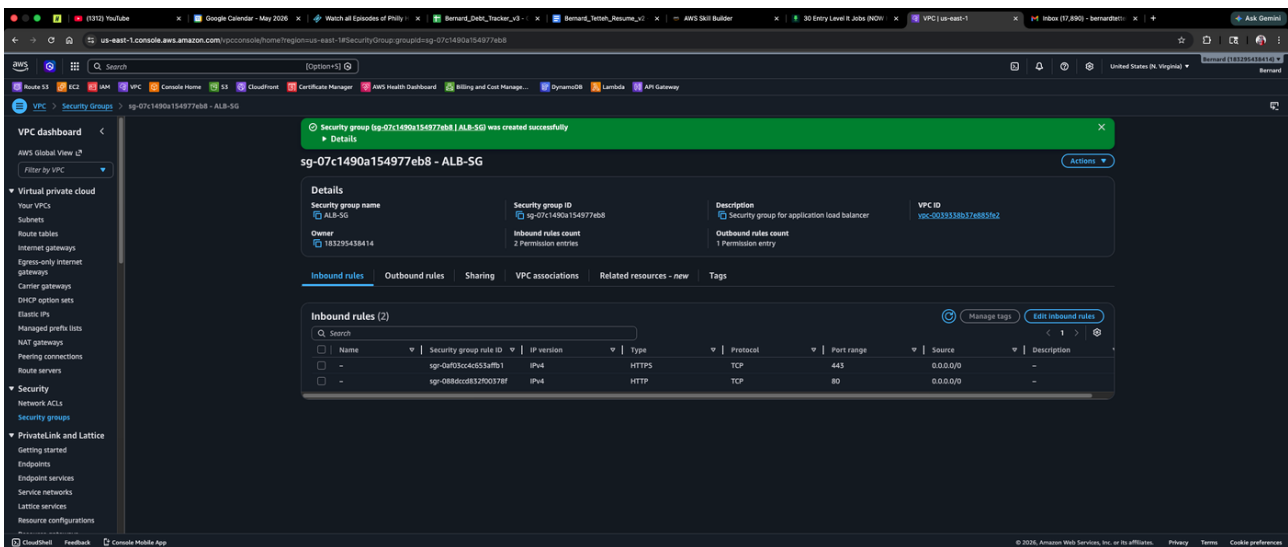


Figure 9 -- ALB-SG Inbound Rules: HTTPS (443) and HTTP (80) from 0.0.0.0/0

3.2 EC2 Security Group (EC2-SG)

EC2 instances only accept port 80 traffic sourced from the ALB security group -- never directly from the internet. This security group chaining ensures all traffic flows through the load balancer.

Direction	Protocol	Port	Source	Purpose
Inbound	TCP	80	ALB-SG (sg-07c1490a...)	ALB health checks and traffic
Outbound	All	All	0.0.0.0/0	Allow all outbound

3.3 RDS Security Group (RDS-SG)

The most restrictive layer -- only EC2 instances can connect to the database on MySQL port 3306. No internet or ALB access to the database is possible.

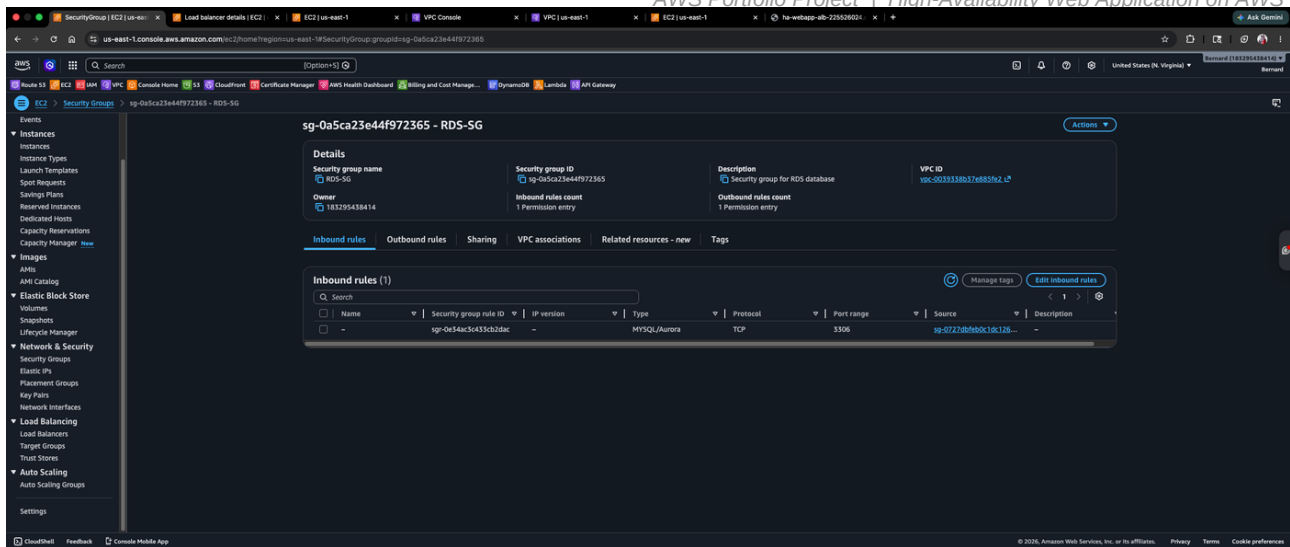


Figure 10 -- RDS-SG Inbound Rules: MySQL/Aurora TCP 3306 from EC2-SG (sg-0727dbfeb0c1dc126) only

4. Compute Configuration

4.1 Launch Template (HA-WebApp-LT)

The Launch Template standardizes every EC2 instance launched by the Auto Scaling Group. It specifies the AMI, instance type, security group, and a User Data bootstrap script that automatically installs Apache httpd on first boot.

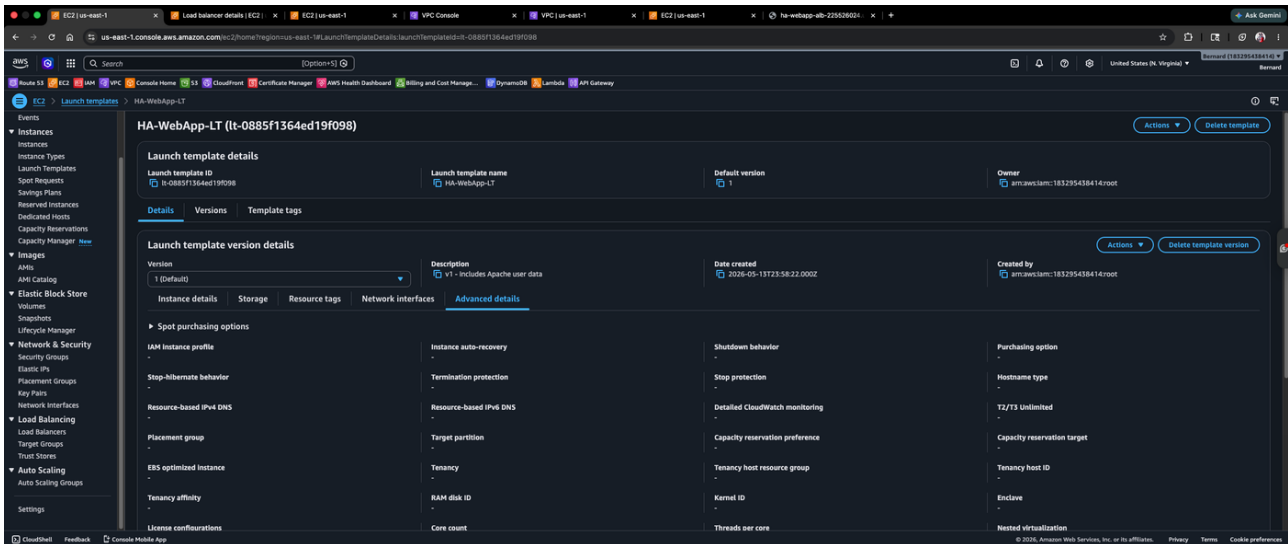


Figure 11 -- HA-WebApp-LT: Version 1, description "V1 - Includes Apache user data", created 2026-05-13

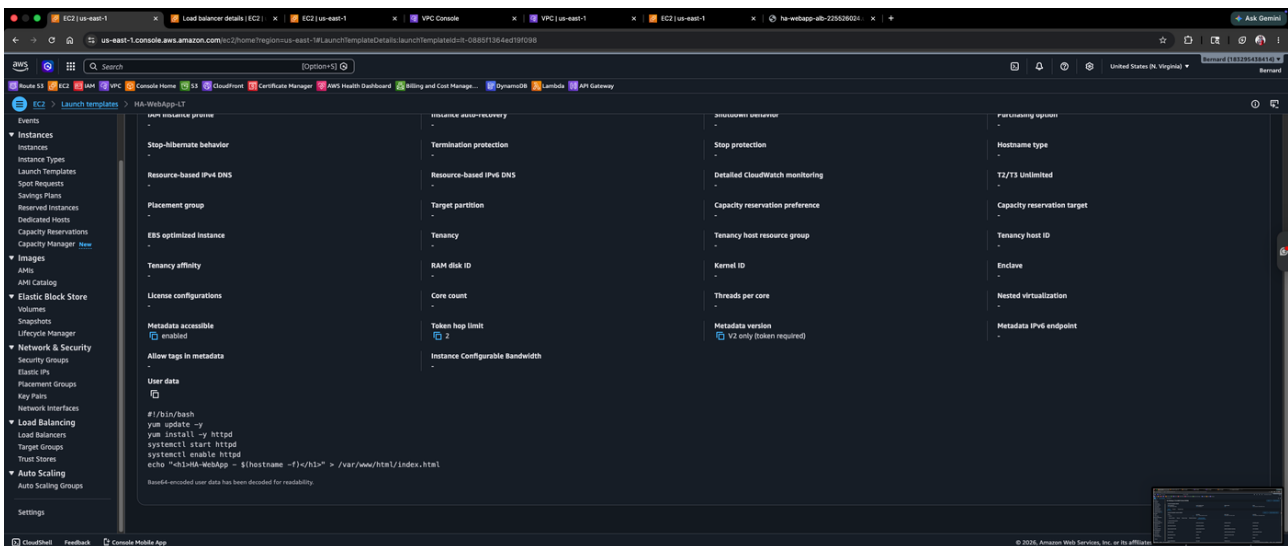


Figure 12 -- User Data script in Advanced Details: installs httpd, starts and enables the service, creates index.html with the instance hostname

The User Data script that runs automatically on every instance launch:

```
#!/bin/bash
yum update -y
yum install -y httpd
systemctl start httpd
```

```
systemctl enable httpd
echo "<h1>HA-WebApp - $(hostname -f)</h1>" > /var/www/html/index.html
```

4.2 Auto Scaling Group (HA-WebApp-ASG)

The ASG uses the Launch Template to maintain exactly 2 running instances at all times -- one in each AZ. It automatically replaces any instance that fails EC2 or ELB health checks within 300 seconds.

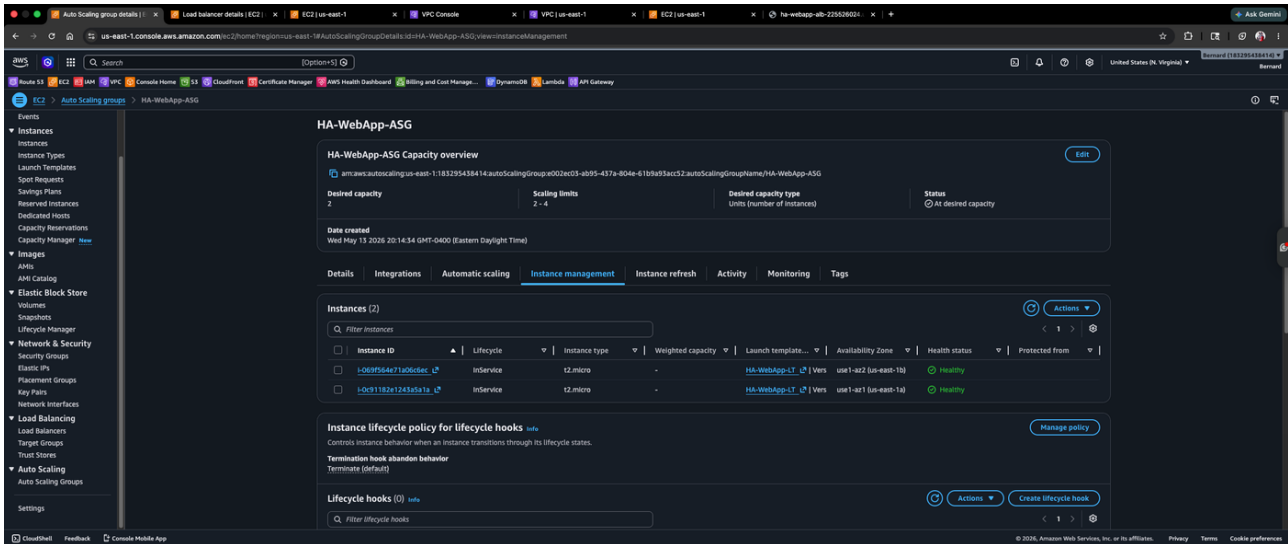


Figure 13 -- ASG Instance Management: 2 instances InService, both Healthy -- one in us-east-1b (AZ-B), one in us-east-1a (AZ-A)

ASG Name	HA-WebApp-ASG
Launch Template	HA-WebApp-LT (Latest version)
Desired / Min / Max	2 / 2 / 4
Health Check Types	EC2 + ELB (Elastic Load Balancing)
Health Check Grace Period	300 seconds
Availability Zones	us-east-1a (Private-Subnet-AZ-A) us-east-1b (Private-Subnet-AZ-B)

4.3 EC2 Instances

Two t2.micro instances run simultaneously at all times -- one per AZ. No public IP addresses are assigned; all inbound access is exclusively through the ALB.

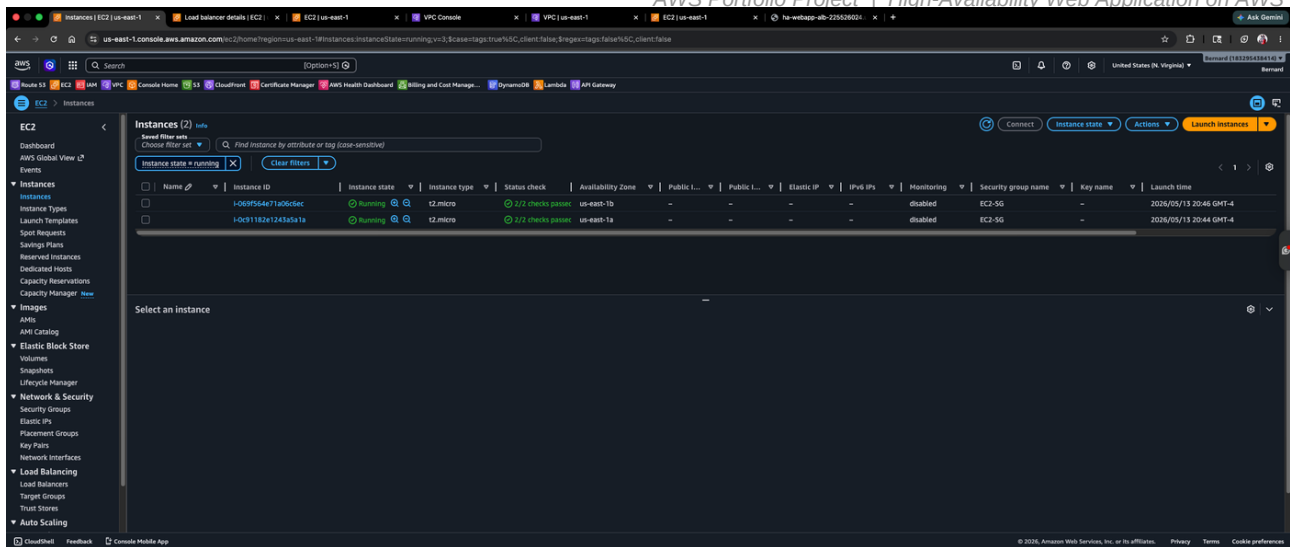


Figure 14 -- EC2 Instances: 2 Running, both passing 2/2 status checks, EC2-SG attached, one in us-east-1b and one in us-east-1a

5. Load Balancer Configuration

The Application Load Balancer (HA-WebApp-ALB) is internet-facing and deployed across both public subnets. It distributes HTTP traffic to healthy EC2 instances using round-robin load balancing.

ALB Name	HA-WebApp-ALB
Type	Application Load Balancer (Layer 7)
Scheme	Internet-facing
Subnets	Public-Subnet-AZ-A (10.0.1.0/24) Public-Subnet-AZ-B (10.0.2.0/24)
Security Group	ALB-SG -- HTTP (80) + HTTPS (443) from 0.0.0.0/0
DNS Name	HA-WebApp-ALB-225526024.us-east-1.elb.amazonaws.com
Listener	HTTP:80 -- Forward to HA-WebApp-TG (100%)
Target Group	HA-WebApp-TG -- Instance type, port 80, HTTP health checks

5.1 Security Group Chaining (Traffic Flow)

Step 1 -- Internet to ALB

- User sends HTTP request to ALB DNS name
- ALB-SG permits HTTP (80) and HTTPS (443) from 0.0.0.0/0
- ALB receives request and selects a healthy registered target

Step 2 -- ALB to EC2 (private subnet)

- EC2-SG permits port 80 inbound ONLY from ALB-SG -- never directly from the internet
- EC2 instances are in private subnets with no public IP addresses assigned
- All traffic must pass through the ALB -- no direct instance access is possible

Step 3 -- EC2 to RDS (private subnet)

- RDS-SG permits MySQL (3306) inbound ONLY from EC2-SG
- Database layer has zero internet exposure -- most restrictive tier

6. Database -- RDS MySQL Multi-AZ

The RDS instance runs MySQL 8.4 with Multi-AZ enabled. AWS automatically maintains a synchronous standby replica in the second Availability Zone. In the event of primary failure or AZ outage, AWS promotes the standby with no manual intervention required -- typical failover time is 1-2 minutes.

DB Identifier	ha-webapp-db
Engine	MySQL Community 8.4
Instance Class	db.t3.micro
Storage	20 GB General Purpose SSD (gp2)
Multi-AZ	Yes -- automatic synchronous standby replica in second AZ
Primary AZ	us-east-1b
Subnet Group	ha-webapp-db-sg (Private-AZ-A 10.0.3.0/24 + Private-AZ-B 10.0.4.0/24)
Security Group	RDS-SG -- MySQL (3306) inbound from EC2-SG only

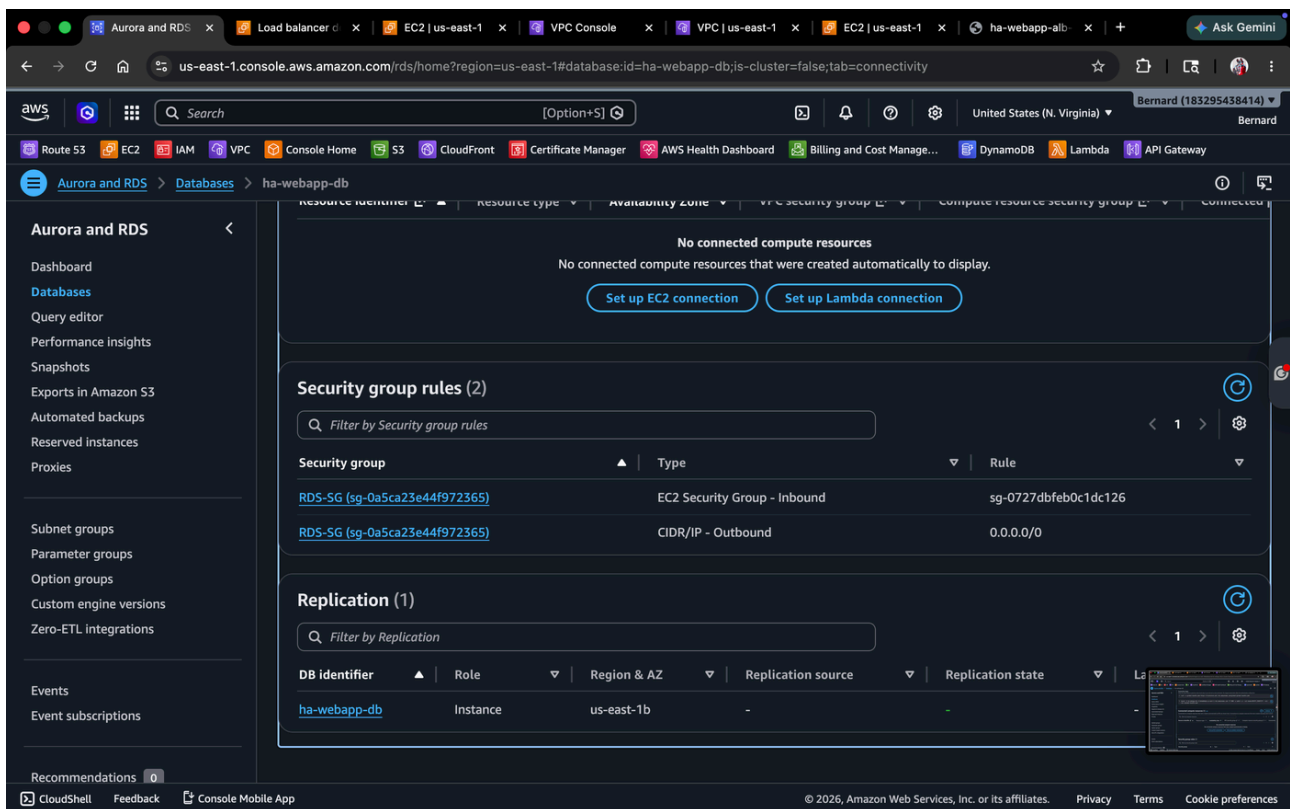


Figure 15 -- RDS Connectivity: RDS-SG rules (inbound from EC2-SG, outbound 0.0.0.0/0), Replication showing ha-webapp-db instance in us-east-1b

7. Proof of Functionality -- Live Application

The following screenshots are the definitive proof that the architecture works end-to-end. The ALB DNS name serves live traffic, and the hostname changes on refresh -- confirming load balancing is active across both Availability Zones.

7.1 Load Balancing Confirmed Across Both AZs

Both screenshots below use the exact same ALB DNS URL. Only the page was refreshed between captures. The different hostnames prove requests are being distributed to two separate EC2 instances in two separate private subnets.

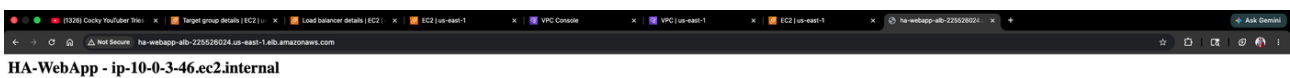


Figure 16 -- Request 1: Served by ip-10-0-3-46.ec2.internal (Private-Subnet-AZ-A, us-east-1a, 10.0.3.x)

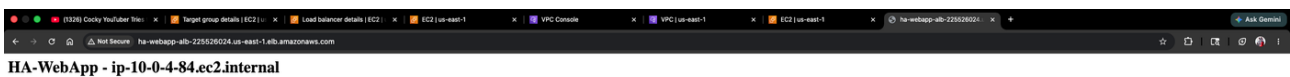


Figure 17 -- Request 2 (same URL, refreshed): Served by ip-10-0-4-84.ec2.internal (Private-Subnet-AZ-B, us-east-1b, 10.0.4.x)

The IP difference (10.0.3.x vs 10.0.4.x) maps directly to Private-Subnet-AZ-A and Private-Subnet-AZ-B -- both AZs are actively serving requests in real time.

7.2 Infrastructure Health Summary

Component	Detail	Status
VPC	HA-WebApp-VPC -- 10.0.0.0/16 us-east-1	Active
Internet Gateway	HA-WebApp-IGW -- Attached to VPC	Attached
NAT Gateway	HA-WebApp-NAT -- Public-Subnet-AZ-A	Available
Public Route Table	0.0.0.0/0 -> IGW -- both public subnets	Active
Private Route Table	0.0.0.0/0 -> NAT GW -- both private subnets	Active
Load Balancer	HA-WebApp-ALB -- Internet-facing, HTTP:80	Active
Target Group	HA-WebApp-TG -- 2/2 Healthy instances	Healthy
EC2 Instance 1	i-069f564e71a06c6ec -- us-east-1b -- 2/2 checks	Running
EC2 Instance 2	i-0c91182e1243a5a1a -- us-east-1a -- 2/2 checks	Running
Auto Scaling Group	HA-WebApp-ASG -- Desired=2, At capacity	Active
RDS Database	ha-webapp-db -- MySQL Multi-AZ -- us-east-1b	Available

8. Challenges & Lessons Learned

This project involved real-world troubleshooting that demonstrates practical AWS diagnostic skills beyond following a setup guide.

Challenge 1 -- Blackhole NAT Gateway Route

Problem

All instances showed Unhealthy in the Target Group despite correct security group configuration. Apache never installed on the instances.

Root Cause

A previous NAT Gateway had been deleted and recreated with a new ID. The Private-RT still pointed 0.0.0.0/0 to the old deleted gateway -- shown as Blackhole status. Private subnet instances had no internet path and silently failed to run "yum install httpd".

Resolution

- Identified Blackhole route in VPC -> Route Tables -> Private-RT
- Updated 0.0.0.0/0 target to the new NAT Gateway ID
- Terminated stale instances -- ASG relaunched fresh ones with working internet access
- New instances installed Apache successfully and health checks passed

Challenge 2 -- ALB in Wrong Subnets (502 Bad Gateway)

Problem

After health checks turned green, the ALB DNS name returned a 502 Bad Gateway error.

Root Cause

The ALB had been associated with the private subnets instead of the public subnets. An internet-facing ALB must reside in public subnets with a route to the Internet Gateway.

Resolution

- Identified public vs private subnets by checking Route Table (IGW route present = public)
- Edited ALB Network Mapping to use Public-Subnet-AZ-A and Public-Subnet-AZ-B
- 502 errors resolved immediately after saving the subnet change

Key Takeaways

- **NAT Gateway route tables are critical -- a Blackhole route silently breaks all private subnet outbound connectivity**

- **Internet-facing ALBs must always be placed in public subnets with IGW routes**
- **User Data scripts are the correct pattern to bootstrap software on Auto Scaling instances**
- **Health check grace periods (300s) prevent premature termination during instance initialization**
- **Security group chaining (ALB-SG as source for EC2-SG) is more secure than CIDR-based rules**

9. AWS Services Used

Service	Purpose
Amazon VPC	Virtual network isolation and subnet segmentation across two AZs
EC2 Auto Scaling	Automatic instance management, replacement, and capacity scaling
Application Load Balancer	Layer 7 HTTP traffic distribution across healthy EC2 instances
Amazon EC2	Web server compute -- Amazon Linux 2023 + Apache httpd
Amazon RDS	Managed MySQL 8.4 with automatic Multi-AZ failover
NAT Gateway	Outbound internet access for private subnet instances
Internet Gateway	Inbound internet access entry point for the VPC
Security Groups	Stateful firewall rules at the ALB, EC2, and RDS tiers
EC2 Launch Templates	Standardized instance configuration with User Data bootstrap
Target Groups	Health monitoring and ALB routing destination

10. Potential Enhancements

- **HTTPS / TLS -- AWS Certificate Manager + HTTPS listener + HTTP-to-HTTPS redirect**
 - o Required for HIPAA compliance -- prevents unencrypted transmission of health data
- **Amazon CloudWatch -- Alarms on CPU, ALB 5xx error rate, and RDS connection count**
- **Auto Scaling Policies -- Target tracking on CPU utilization for dynamic scale-in/out**
- **AWS WAF -- Web Application Firewall for SQL injection and XSS protection**
- **Infrastructure as Code -- CloudFormation or Terraform for repeatable, version-controlled deployments**
- **CI/CD Pipeline -- AWS CodePipeline + CodeDeploy for automated zero-downtime deployments**

Document Complete

HA-WebApp Portfolio Project -- Bernard Tetteh -- May 2026

Built with AWS Console | us-east-1 (N. Virginia) | Amazon Web Services